

Tyler Vander Mooren

CYB 410 – Secure Software

Professor Linda Hamons

February 17, 2026

Reflection - “goto fail”

What Happened?

When I first ran the test suite, the tests for buggy bad chain and bad hostname failed. Not sure if intended and I missed something, but in other words the buggy function should incorrectly treat certificates with `chain_ok = false` or `hostname_ok = false` as valid. Seeing those failures made it clear however, just how sensitive control flow is in security code. Even small structural differences completely change whether bad certificates are acceptable or rejected.

Why did it happen?

The issue simulates Apple’s original stray goto fail; vulnerability, where an unconditional jump caused critical validation checks to be skipped. In this case, the flawed control flow allows the execution to bypass certificate chain and hostname verification. Because those checks are skipped, the function can return success even when the certificate is not fully validated. This shows just how one misplaced control statement can undermine an entire security mechanism.

How did you fix it?

My implementation involved removing any unconditional control flow and ensuring that all validation checks execute sequentially. The function validates that the input is a dictionary containing the required keys. It then verifies that each value is strictly Boolean and only returns True if all three fields are exactly True. Any missing key, malformed input or failed validation immediately results in False, enforcing a fail-closed design.

Secure coding lessons

This exercise reinforces just how dangerous unclear control flow can be in security-critical code. Some lessons learned are to avoid unconditional jumps or duplicated logic that can accidentally bypass checks. Always validate input structure and data types explicitly. That automated testing is essential because subtle logic errors are not always obvious when inspection is done. Also, code reviews and consistent formatting also reduce the risk of introducing this class of bugs.

Advice for beginners

Some advice for beginners is to keep the control flow simple and readable. Another thing is do not rely on flags or complicated branching when validating security conditions. Always check inputs before using them and write tests for edge cases. The biggest advice is that even a single misplaced statement in authentication or validation logic can create a serious vulnerability, so proceed with caution.

screenshotsuccess1.png

```
Command Prompt
X + -
FAILED tests/test_goto_fail.py::test_buggy_accepts_bad_hostname - AssertionError: BUG: bad hostname was accepted!
2 failed, 4 passed in 0.11s

(venv) C:\Users\tyler\Desktop\spring\cyb410\week6\goto_fail_python_starter>python -m pytest -q
FF..... [100%]
===== FAILURES =====
_____ test_buggy_accepts_bad_chain _____

    def test_buggy_accepts_bad_chain():
>     assert verify_cert_buggy(bad_chain) is True, "BUG: bad chain was accepted!"
E     AssertionError: BUG: bad chain was accepted!
E     assert False is True
E     + where False = verify_cert_buggy({'chain_ok': False, 'hostname_ok': True, 'sig_ok': True})

tests/test_goto_fail.py:10: AssertionError
_____ test_buggy_accepts_bad_hostname _____

    def test_buggy_accepts_bad_hostname():
>     assert verify_cert_buggy(bad_host) is True, "BUG: bad hostname was accepted!"
E     AssertionError: BUG: bad hostname was accepted!
E     assert False is True
E     + where False = verify_cert_buggy({'chain_ok': True, 'hostname_ok': False, 'sig_ok': True})

tests/test_goto_fail.py:13: AssertionError
===== short test summary info =====
FAILED tests/test_goto_fail.py::test_buggy_accepts_bad_chain - AssertionError: BUG: bad chain was accepted!
FAILED tests/test_goto_fail.py::test_buggy_accepts_bad_hostname - AssertionError: BUG: bad hostname was accepted!
2 failed, 7 passed in 0.11s

(venv) C:\Users\tyler\Desktop\spring\cyb410\week6\goto_fail_python_starter>
```